December 19, 2025

C964: Computer Science Capstone

Part A: Letter of Transmittal

December 17, 2025

Chief Information Officer

Enterprise Security Operations

123 Main Street

Atlanta, GA 30303

Dear Chief Information Officer,

Phishing emails remain one of the most common and effective attack vectors used to compromise organizational systems. Despite continued investment in email filtering and security awareness training, many organizations still rely heavily on manual review and user reporting to identify phishing attempts. This process is time-consuming, inconsistent, and increasingly impractical as the volume and sophistication of phishing campaigns continue to grow. Delays in detection significantly increase the risk of credential compromise, malware delivery, and downstream security incidents.

To address this challenge, I recommend the adoption of a machine-learning-based phishing email classification application designed to assist security and IT teams in rapidly identifying potentially malicious emails. The application allows users to input the text of an email and receive an immediate classification indicating whether the message is likely safe or phishing-related. By leveraging supervised machine learning techniques trained on real-world phishing data, the solution provides faster and more consistent triage while complementing existing security workflows.

The primary benefit of this solution is the reduction of manual effort required to review reported emails. By automating the initial classification process, security personnel can

prioritize higher-risk messages and focus their attention on investigation and response rather than routine screening. This approach improves response times, reduces analyst fatigue, and supports more consistent decision-making across the organization. Because the application is built using open-source tools and operates locally, implementation costs remain minimal while allowing for future scalability or integration into broader security operations.

The estimated cost to develop and deliver a functional prototype of this application is $2,000, representing development time and testing. The projected development timeline for the prototype is two weeks, allowing the organization to quickly evaluate effectiveness and determine whether further enhancements or integration efforts are warranted. The application requires no specialized hardware and can be deployed on standard systems using widely adopted Python libraries.

Ethical considerations, including the risk of false positives or false negatives, have been addressed by positioning the application as a decision-support tool rather than a replacement for human judgment. The solution is intended to assist analysts during initial triage, not to autonomously block or remediate emails without review.

My academic background in computer science, combined with hands-on experience in Python development, data analysis, and cybersecurity fundamentals, provides a strong foundation for the successful development and evaluation of this solution. This project demonstrates the practical application of machine learning to a real-world security problem and offers a cost-effective method for enhancing phishing detection capabilities.

Thank you for your time and consideration. I welcome the opportunity to discuss this recommendation further and explore how this solution could support your organization’s security objectives.

Sincerely,

Jamal Copeland

Jamal Copeland, Cyber Security Engineer

Part B: Project Proposal Plan

Project Summary

Phishing emails continue to be a leading cause of security breaches across organizations of all sizes. Despite advancements in email filtering and security awareness training, many organizations still depend on manual review and user reporting to identify suspicious messages. This approach is inefficient, inconsistent, and increasingly difficult to scale as phishing attacks become more frequent and sophisticated. The purpose of this project is to design and implement a machine learning–based phishing email classification application that supports faster and more consistent identification of phishing attempts.

The proposed solution is a supervised machine learning application that analyzes the textual content of emails and classifies them as either *phishing* or *safe*. By automating the initial triage process, the application supports security teams by reducing manual workload and enabling analysts to focus on higher-risk cases. This decision-support tool improves response times and contributes to more effective security operations without replacing human judgment.

Data Summary

The dataset used for this project consists of labeled phishing and legitimate email text sourced from a publicly available dataset on Kaggle. The dataset includes email body content along with classification labels indicating whether each email is phishing or safe. This data is suitable for supervised machine learning because it provides real-world examples of both malicious and legitimate communication patterns.

The data is processed using the ETL (Extract, Transform, Load) approach. During the extraction phase, the email text and labels are loaded into the application using the pandas library. The transformation phase includes data cleaning and preprocessing to prepare the text for machine learning. This includes removing missing values, normalizing text, converting all text to lowercase, removing punctuation and special characters, and transforming the text into numerical features using vectorization techniques. The cleaned and transformed data is then loaded into the machine learning pipeline for training and testing.

Because the dataset contains email text, care is taken to ensure ethical data handling. The dataset does not include personally identifiable information (PII) and is used strictly for academic and analytical purposes. Any example inputs provided in the application are generic and do not represent real individuals.

Implementation

The development of the phishing email classification application follows a structured machine-learning lifecycle designed to support reliable analysis, model training, and decision-support deployment. The process begins with acquiring a labeled dataset containing both phishing and legitimate email messages. This data is reviewed to ensure label consistency and usability, with

incomplete or invalid records removed. The dataset is then divided into training and testing subsets to allow for objective performance evaluation.

Once the data is prepared, exploratory analysis is conducted to better understand the underlying patterns within phishing and safe emails. This exploration includes reviewing class distribution and identifying common linguistic characteristics associated with phishing attempts. Examining the dataset at this stage helps identify potential biases, such as class imbalance or repetitive content, and informs decisions made during preprocessing and feature engineering.

Following exploration, the email text is cleaned and transformed to improve machine-learning performance. Text is standardized by normalizing case, removing noise, and handling irregular formatting. The cleaned email content is then converted into numerical feature representations using a vectorization technique that captures the relative importance of words across messages. This transformation enables the classification algorithm to identify meaningful distinctions between phishing and legitimate emails.

With features prepared, a supervised learning model is trained to classify emails based on their textual content. The model learns patterns that distinguish phishing messages from safe communications by analyzing the relationship between extracted features and known labels. Model parameters are adjusted as needed to improve generalization and reduce overfitting. Once training is complete, the model is preserved for use within the application.

The trained model is evaluated using a holdout testing approach to verify its effectiveness on unseen data. Performance is measured using standard classification metrics, including accuracy, precision, recall, and F1-score. Particular attention is given to balancing false positives and false negatives, as misclassification in either direction carries operational risk. Evaluation results guide any necessary refinements before deployment.

Finally, the model is integrated into a user-facing application that allows users to input email text and receive a classification indicating whether the message is likely phishing or safe. Input validation is implemented to prevent application errors when users submit empty or incomplete data. The application is positioned as a decision-support tool that enhances analyst efficiency by accelerating initial triage, while preserving human oversight for final determination.

Timeline

The proposed timeline outlines a structured and sequential approach to developing the phishing email classification application, ensuring each phase builds logically upon the completion of the previous tasks. The schedule begins with proposal approval and data source identification, followed by data preparation and exploratory analysis to establish a strong foundation for model training. Model development, evaluation, and refinement are scheduled after data preprocessing to ensure accuracy and reliability before application integration. The final phases focus on application testing, validation, documentation, and submission, allowing sufficient time to verify functionality and performance. This timeline reflects a realistic development pace for a single developer and supports timely delivery of a functional machine learning–based decision-support tool.

Milestone or Deliverable	Dependency	Duration (Days)	Projected Start Date	Anticipated End Date	Resource Assigned
Acceptance of the project proposal	None	1 Day	01/13/2026	01/13/2026	Developer
Identify dataset sources and technology stack	Proposal acceptance	1 Day	01/14/2026	01/14/2026	Developer
Collect phishing and safe email datasets	Data source identification	1 Day	01/15/2026	01/15/2026	Developer
Clean and preprocess email text data	Dataset collection	2 Days	01/16/2026	01/17/2026	Developer
Perform exploratory data analysis (EDA) to identify patterns and class distribution	Data preprocessing	2 Days	01/18/2026	01/19/2026	Developer
Transform email text into numerical features for model training	Completion of EDA	1 Day	01/20/2026	01/20/2026	Developer
Train supervised machine learning model for phishing classification	Feature engineering complete	2 Days	01/21/2026	01/22/2026	Developer
Evaluate and fine-tune the classification model	Initial model training	2 Days	01/23/2026	01/24/2026	Developer
Develop a functional prototype with user input and classification output	Trained model	2 Days	01/25/2026	01/26/2026	Developer
Conduct application testing and handle edge cases (empty input, short text)	Prototype development	2 Days	01/27/2026	01/28/2026	Developer
Validate model accuracy and generate performance metrics	Application testing	1 Day	01/29/2026	01/29/2026	Developer
Finalize application and supporting documentation	Validation complete	2 Days	01/30/2026	01/31/2026	Developer
Prepare final submission and review materials	Documentation complete	1 Day	02/01/2026	02/01/2026	Developer

Evaluation Plan

The effectiveness of the phishing email classification application will be evaluated using both quantitative model performance metrics and functional application testing. Model evaluation focuses on verifying that the supervised learning algorithm accurately distinguishes between phishing and legitimate emails while maintaining consistent performance on unseen data. A holdout validation approach will be used, where a portion of the labeled dataset is reserved for testing after the model has been trained. This method ensures that evaluation results reflect the model's ability to generalize beyond the training data.

Model performance will be measured using standard classification metrics, including accuracy, precision, recall, and F1-score. Accuracy provides an overall measure of correct classifications, while precision and recall offer deeper insight into the tradeoffs between false positives and false negatives. These metrics are particularly important in phishing detection, as excessive false positives can increase analyst workload, while false negatives may allow malicious emails to bypass detection. A confusion matrix and classification report will be used to analyze these outcomes and confirm that performance aligns with the project's decision-support objectives.

In addition to model-level evaluation, the application itself will be tested to ensure reliability and usability. Functional testing will verify that the application correctly processes user-submitted email text and consistently returns classification results. Edge-case testing will be conducted to confirm that the application handles empty inputs, extremely short messages, and invalid submissions without crashing or producing misleading output. These tests ensure that the application behaves predictably in real-world usage scenarios.

The evaluation process will be considered successful if the model demonstrates acceptable classification performance on the test dataset and the application consistently provides stable, interpretable results. Evaluation findings will be documented and used to guide any final refinements prior to submission. This evaluation approach ensures that the developed data product meets both technical performance requirements and practical decision-support needs.

Costs

The development and implementation of the phishing email classification application require minimal resources due to the use of open-source software and publicly available data. All development activities are conducted on a standard personal computer, eliminating the need for specialized hardware or additional infrastructure. The software environment consists of widely used, no-cost tools, including Python, Jupyter Notebook, and common machine learning libraries, which are sufficient to support data preprocessing, model training, evaluation, and application development.

The data set used for this project is obtained from a public Kaggle repository at no cost. Development is completed by a single developer, and the primary expense associated with the project is the time required to design, train, test, and integrate the machine learning model into a functional decision-support application. Based on an estimated forty hours of development at a

rate of $50 per hour, the total projected cost for the prototype is approximately $2,000. This cost-effective approach enables the organization to assess the feasibility and effectiveness of the solution before committing to further investment or integration into production environments.

Resource	**Description**	**Cost**
Hardware	Personal computer	$0
Software	Python, Jupyter Notebook, pandas, scikit-learn	$0
Data	Public phishing email dataset (Kaggle)	$0
Personnel	Developer time (40 hours @ $50/hour)	$2,000
Total Estimated Cost		**$2,000**

Part C: Application

Required Files:

- **capstone.ipynb**: Contains the source code for the machine-learning–based phishing email classification application, including data preprocessing, model training, evaluation, and user input functionality.
- **Phishing_Email.csv**: The raw dataset is used to train and evaluate the phishing email classification model.

Part D: Post-implementation Report

Solution Summary

The objective of this project was to address the challenge of detecting and managing phishing emails, which remain a common and effective attack vector against organizations. Manual review of suspicious emails is inefficient and inconsistent, creating the need for an automated decision-support solution that can assist security teams during initial email triage.

To solve this problem, a machine-learning–based phishing email classification application was developed. The solution consisted of a command-line interface (CLI) application that analyzed user-submitted email text and classified messages as either phishing or safe using a supervised learning approach. Email content was transformed into numerical features using TF-IDF vectorization and classified using a Support Vector Machine (SVM).

The application successfully classified phishing and legitimate emails in real time and provided consistent classification results suitable for decision-support use. The implemented solution fulfilled the requirements defined in Parts A and B by delivering a reliable, cost-effective phishing detection application that operates locally and supports analyst decision-making.

Data Summary

The raw data used in this project consisted of labeled phishing and legitimate email messages sourced from a publicly available CSV dataset included in the project repository. The dataset contained email body text along with classification labels indicating whether each message was phishing or safe. This data provided real-world examples of common phishing techniques and legitimate communications, making it suitable for supervised machine learning.

During development, the dataset was imported into the application using the pandas library and processed to ensure consistency and accuracy. Records containing missing or invalid email content were removed, and text data was standardized to prepare it for feature extraction and classification. Input handling and validation were also applied during application execution to maintain data integrity when users submitted email text for classification.

The cleaned and processed data was used throughout model training, evaluation, and testing. These data management steps ensured that the machine learning model operated on reliable, structured input and supported accurate classification results during both training and real-time application use.

Machine Learning

This project implemented a supervised machine learning approach to distinguish phishing emails from legitimate messages based on their textual content. Prior to classification, email text was converted into numerical representations using a TF-IDF vectorization technique, allowing the

model to weigh the significance of terms appearing across different messages. This process enabled the system to recognize patterns commonly associated with phishing attempts, such as urgency-based language or requests for sensitive information.

A Support Vector Machine (SVM) classifier was trained using labeled examples of phishing and safe emails. The dataset was prepared through preprocessing steps that ensured consistency and removed incomplete records. To evaluate generalization performance, the data was divided into separate training and testing sets before model training.

The SVM algorithm was selected due to its effectiveness in handling high-dimensional feature spaces produced by text vectorization and its reliability in binary classification tasks. This supervised learning framework allowed the model to learn distinguishing characteristics from real-world email data and apply those learned patterns to previously unseen inputs. As a result, the model provided consistent and interpretable classification results suitable for decision-support use.

Validation

The phishing email classification model was validated as a supervised learning system using labeled data with known outcomes. Model performance was evaluated on a holdout test set using standard classification metrics, including accuracy, precision, recall, and F1-score. These metrics were selected to evaluate the model's ability to accurately identify phishing emails while minimizing both false positives and false negatives.

Evaluation results demonstrated strong overall performance, with balanced precision and recall across both phishing and legitimate email classes, and an overall accuracy of approximately 96% on unseen test data. The classification report and confusion matrix generated during evaluation provided additional insight into model behavior and confirmed consistent performance across classes.

Functional testing was also conducted by submitting a variety of email inputs through the application, including phishing examples, legitimate messages, and edge cases. The combined validation results indicated that the model produced reliable and interpretable classifications suitable for use as a decision-support tool during initial email triage.

Visualizations

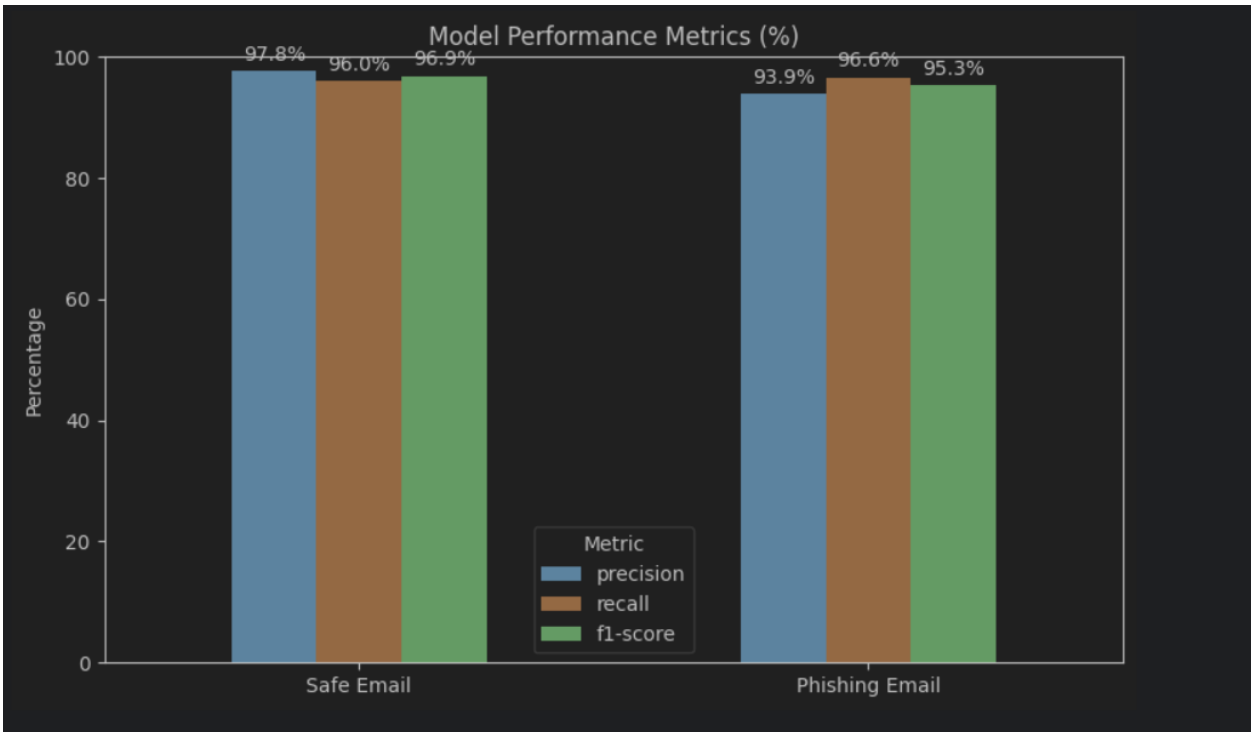
Three unique visualizations were created within the Jupyter Notebook environment to support data exploration and model evaluation for the phishing email classification application. Each visualization was designed to highlight a different aspect of the dataset and the model's behavior.

The first visualization was a bar chart illustrating the distribution of phishing and legitimate emails within the dataset. This visualization provided insight into class balance and confirmed that sufficient examples were available for supervised learning.

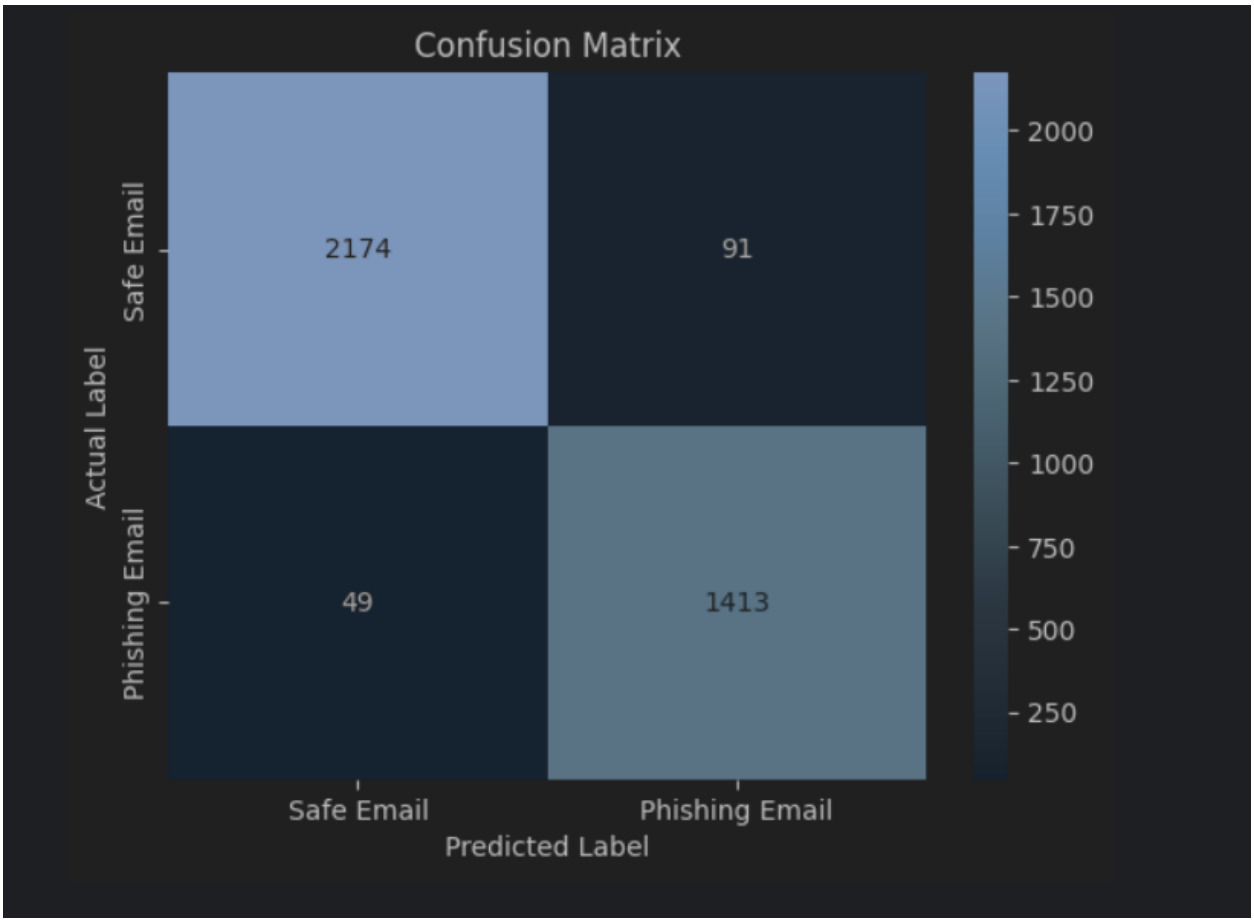
The second visualization was a bar chart summarizing the model's classification outcomes, showing how frequently emails were categorized as phishing or safe during evaluation. This visualization helped illustrate overall prediction trends and supported the interpretation of model behavior.

The third visualization was a confusion matrix, which compared predicted classifications against actual labels. This visualization provided a clear representation of model accuracy, highlighting instances of false positives and false negatives, which supported the assessment of the model's effectiveness as a decision-support tool.

Distribution of email types in Jupyter Notebook "Data Exploration and Visualization"



Precision of Safe and Phishing email accuracy Jupyter Notebook “Model Evaluation”



Jupyter Notebook Confusion Matrix (Located in Model Evaluation)

User Guide — Phishing Email Classification Application

(Recommended) – On Web Use

1. User Guide Navigate to the following Google Colab https://colab.research.google.com/
2. Click “Upload,” then “Browse,” and find the file named capstone.ipynb and click “Open.”
3. Select the folder icon on the left-hand side of the page.
4. Upload the provided dataset “Phishing_Email.csv”
5. At the top of the screen, click the “Runtime” tab and select “Run All.”
6. This will run through each code block automatically.
7. Follow the prompt to enter an email.
8. Enter an email body… some have been suggested
9. The model will return a value indicating whether the email is Phishing or Safe
10. To run again, run the last cell

Alternatively,

Software Requirements

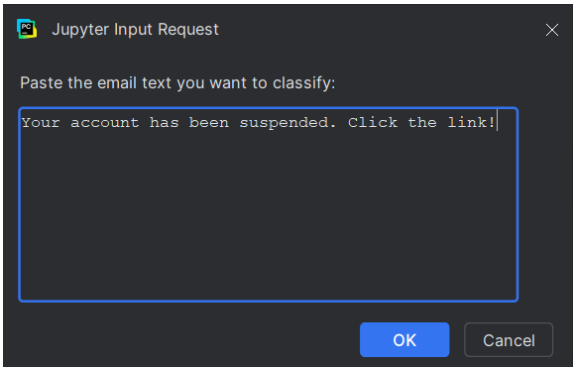
1. Python 3.10+
2. Jupyter Notebook
3. Required Python libraries:
    - pandas
    - scikit-learn
    - numpy

Installation & Setup

1. Download and unzip the project files.
2. Open Jupyter Notebook.
3. Open `capstone.ipynb`.
4. Run all cells from top to bottom.

Using the Application

5. When prompted, enter the full text of an email.



6. Execute the cell to submit the email for analysis
7. The application will output one of the following:
    - **Phishing Email**
    - **Safe Email**
8. Example Usage**:**
    - **Input: "**Your account has been suspended. Please click the link below to verify your information**."**
    - **Output:** "Phishing Email."

Reference Page

No references were used in the making of this project.